ローカルの GPU 搭載PCで生成AIを動かすガイド（社内・オフライン利用向け）

社外秘の画像を外部に出さずに試したい場合は、社内の GPU 搭載 PC で実行します。方法は2通りあります。

	A. ComfyUI	B. Python（本講座のノートブック）
操作	ノードをつなぐGUI	コードを書く／実行する
向いている人	まず動かして結果を見たい、非エンジニアと共有したい	自動化・システム組込み・評価実験をしたい
最新モデルへの対応	非常に速い（カスタムノード）	ライブラリ対応待ちのことがある
再現性・バッチ処理	ワークフローJSONで共有、API でバッチ可	高い（コードで完全に記述）

---

0. ハードウェアの目安

GPU メモリ（VRAM）	できること	GPU の例	PC 価格の目安（2026年10月）
8 GB	Real-ESRGAN、LaMa、SD1.5（Inpainting / ControlNet）	RTX 5060（RTX 4060）	デスクトップ 約20〜30万円
12〜16 GB	SDXL、LoRA 学習（SD1.5）、VLM 3B、DiffBIR	RTX 5060 Ti 16GB / RTX 5070 Ti	約25〜35万円 / 約36〜50万円
24 GB	SUPIR、FLUX 系（FP8）、VLM 7B	RTX 5090 Laptop（RTX 4090 は生産終了）	ノートPC 約60〜100万円
32 GB 以上	FLUX / Qwen-Image 系をフル精度、動画系	RTX 5090 / RTX PRO 6000	約100〜130万円 / 200万円〜

- 価格は BTO パソコンの目安です。2026年は AI 需要によるメモリ・GPU 不足で値上がりが続いています（RTX 5090 は GPU 単体で約80万円）。購入前に必ず最新価格を確認してください。
- **NVIDIA の GPU を推奨**（CUDA 対応。AMD / Apple Silicon は対応状況にばらつきあり）
- システム RAM は 32GB 以上、SSD の空き 200GB 以上あると安心（モデルは1つ数GB〜20GB）
- ノートPC の GPU はデスクトップ版より VRAM・性能が低いことがあるので型番に注意

A. ComfyUI を使う（Windows）

A-1. インストール（ポータブル版、管理者権限不要）

なぜポータブル版か：Python 本体（`python_embeded`）と PyTorch などのライブラリが ComfyUI のフォルダの中に丸ごと入っています。そのため、パソコンにすでに入っている Python や他のソフトの環境を汚しません。インストール作業が不要で、フォルダを消せば削除でき、フォルダごとコピーすれば別の PC（閉域網など）にも持ち込めます。

1. NVIDIA ドライバを最新にする（NVIDIA App または公式サイト）
2. https://github.com/Comfy-Org/ComfyUI/releases から `ComfyUI_windows_portable_nvidia.7z` をダウンロードし、**日本語や空白を含まないフォルダ**（例 `C:\ComfyUI`）に 7-Zip で展開
3. **ComfyUI-Manager を入れる**（ノード・モデルの管理ツール。展開しただけでは入っていません。どちらか一方の方法で） - 方法A（`ComfyUI\manager_requirements.txt` がある新しい版）：展開したフォルダで PowerShell を開き `.\python_embeded\python.exe -m pip install -r ComfyUI\manager_requirements.txt` を実行し、`run_nvidia_gpu.bat` をメモ帳で開いて1行目の末尾に `--enable-manager` を追加 - 方法B（従来の方法、Git が必要）：`cd ComfyUI\custom_nodes git clone https://github.com/Comfy-Org/ComfyUI-Manager` Git が無い場合は GitHub の「Code → Download ZIP」を展開し、`ComfyUI\custom_nodes\ComfyUI-Manager` という名前で置く
4. `run_nvidia_gpu.bat` をダブルクリック → ブラウザで `http://127.0.0.1:8188` が開く。画面上部に **Manager** ボタンが出ていれば成功（デスクトップアプリ版 ComfyUI Desktop もあります）

A-2. モデルの置き場所

種類	フォルダ
Stable Diffusion 本体（checkpoint）	`ComfyUI\models\checkpoints\`
超解像モデル（Real-ESRGAN など）	`ComfyUI\models\upscale_models\`
ControlNet	`ComfyUI\models\controlnet\`
LoRA	`ComfyUI\models\loras\`

A-3. 配布ワークフローの使い方

1. `comfyui_workflows\*.json` を ComfyUI の画面に **ドラッグ＆ドロップ**
2. 赤枠のノード（足りないカスタムノード）があれば、**Manager → Install Missing Custom Nodes** でインストールし、ComfyUI を再起動
3. 足りないモデルは `comfyui_workflows\README.pdf` の一覧からダウンロードして所定のフォルダへ
4. 足りないライブラリがあれば追加する（次の A-4）
5. 「Load Image」ノードで画像を選び、**Run（Queue Prompt）**

A-4. 不足ライブラリのインストール（ポータブル版）

起動時のログに `IMPORT FAILED` や `No module named 'xxx'` と出たノードは、Python のライブラリが足りません。**必ずフォルダ内の Python（`python_embeded\python.exe`）に対して** pip を実行します（ふつうに `pip install` と打つと、ComfyUI とは別の Python に入ってしまいます）。

`ComfyUI_windows_portable` フォルダで PowerShell を開いて：

```
# いまの PyTorch と CUDA のバージョンを確認
.\python_embeded\python.exe -c "import torch; print(torch.__version__, torch.version.cuda)"

# カスタムノードが必要とするライブラリをまとめて入れる
.\python_embeded\python.exe -m pip install -r ComfyUI\custom_nodes\ComfyUI-SUPIR\requirements.txt

# 個別に入れる例：OpenCV、omegaconf
.\python_embeded\python.exe -m pip install opencv-python omegaconf

# PyTorch を入れ替える例（RTX 50 系は CUDA 12.8 版が必要）
.\python_embeded\python.exe -m pip install --upgrade torch torchvision torchaudio --index-url http
s://download.pytorch.org/whl/cu128

# 入っているライブラリの一覧を保存（トラブル時の記録・復元用）
.\python_embeded\python.exe -m pip list > pip_list_backup.txt
```

- ComfyUI-Manager の「Install PIP packages」からも入れられます。
- **PyTorch の入れ替えは慎重に**：他のノードが動かなくなることがあります。フォルダごとバックアップしてから行ってください。
- `opencv-python` と `opencv-contrib-python` が両方入ると壊れることがあります。どちらか一方にします。
- 本講座の DifFace ノード（`comfyui_workflows\custom_nodes\seminar_nodes`）は、ComfyUI フォルダ内に DifFace のリポジトリが必要です：`cd ComfyUI` → `git clone https://github.com/zsyOAOA/DifFace`

A-5. 社内利用での注意

- ComfyUI 本体は画像を外部に送信しません。ただし **カスタムノードの中には外部APIを呼ぶもの** があります（ComfyUI 公式の「API ノード」は有料クラウドを呼びます）。社内利用ではインストールするノードを限定し、出所を確認してください。
- 起動オプション `--listen` を付けると社内LANの他PCから使えますが、認証がないので社外公開は厳禁です。
- 完全オフラインで使う場合は、必要なモデルとノードを事前にダウンロードしてから切り離します。

B. Python だけで実行する（本講座のノートブックをローカルで）

B-1. Python 環境の作成（uv を使う方法・推奨）

PowerShell で：

```
# uv（高速な Python パッケージ管理ツール）をインストール
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"

# 教材を GitHub から取得して、そのフォルダに入る（日本語・空白を含まない場所を推奨）
#   Git が無い場合：GitHub のページの「Code → Download ZIP」を展開し、C:\genai_seminar という名前で置く
git clone https://github.com/takeshi-nagata-mgu/genai-image-restoration-seminar C:\genai_seminar
cd C:\genai_seminar
uv venv --python 3.12
.venv\Scripts\activate

# PyTorch（CUDA版）を先に入れる。RTX 50 シリーズは cu128 以降が必須
# （講師の PC と同じ版。GPU が RTX 40 系以前なら cu126 でも可）
uv pip install torch==2.11.0 torchvision==0.26.0 --index-url https://download.pytorch.org/whl/cu12
8

# 本講座で使うライブラリ（講師の PC で動作を確認した版。教材フォルダの requirements.txt）
uv pip install -r requirements.txt

# GPU が見えるか確認
python -c "import torch; print(torch.__version__, torch.cuda.is_available(), torch.cuda.get_device
_name(0))"
```

- **PyTorch を必ず先に入れてください。** 先に `requirements.txt` を入れると、PyTorch が CPU 版で入ってしまいます(Windows の場合)。
- 版を指定せずに最新を入れる場合は、`requirements.txt` の代わりに次を実行します(新しい版で動かない場合は、上の指定版に戻してください): `powershell uv pip install diffusers transformers accelerate peft safetensors spandrel lpips torchmetrics piq ` opencv-python-headless scikit-image scipy matplotlib onnx onnxruntime onnxscript psutil jupyterlab`
- conda / pip でも同様にできます。`pip install` の前に PyTorch 公式サイトのコマンドで CUDA 版を入れるのがポイントです。

B-2. ノートブックの実行

```
cd C:\genai_seminar
.venv\Scripts\activate     # 2 回目以降も、毎回この 2 行から
jupyter lab                # ブラウザが開く → notebooks フォルダの .ipynb を開いて上から実行
```

- VS Code の Jupyter 拡張機能でも開けます(カーネルに `.venv` の Python を選択)。
- まず **00** を実行し、GPU 名が表示されることを確認してください。「GPUが見つかりません」と出たら、CPU 版の PyTorch が入っています(B-1 の注意を参照)。
- セルは **上から順に** 実行します。途中から実行したり、カーネルを再起動したあとに途中のセルだけ実行したりすると、`NameError`(変数が無い)になります。その場合は[Kernel]→[Restart Kernel and Run All Cells]で最初から実行し直してください。

- 学習済みの重みはノートブックと同じフォルダの `weights`（`C:\genai_seminar\notebooks\weights`）に、Hugging Face のモデルは B-3 の場所に、初回だけダウンロードされます。
- Colab 専用の部分（Google Drive への保存など）は、ローカルでは自動的に飛ばします。ノートブック 15（SUPIR）はローカルでも動きますが VRAM 24GB 以上が必要です。ローカルでは A. の ComfyUI で `sr_SUPIR.json` を使う方が手軽です。
- Mac（Apple Silicon）では、セットアップのセルの `DEVICE = "cuda" if ... else "cpu"` を `DEVICE = "mps"` に書き換えてください（書き換えないと CPU で動き、非常に遅くなります）。
- 教材の更新を取り込むときは `git pull`（ZIP の場合は取り直し）。自分で書き換えたノートブックは、別名で保存しておくと上書きされません。

B-2b. Hugging Face のトークン（HF_TOKEN）

本講座のモデル（Stable Diffusion、ControlNet、Qwen-VL、Depth Anything など）は、**トークンなしでダウンロードできます**。トークンを設定していないと次のような警告が出ますが、無視して構いません：`The secret HF_TOKEN does not exist ...`（Colab）／`You are sending unauthenticated requests to the HF Hub ...`（ローカル）

トークンを設定すると、ダウンロードの回数制限が緩くなり、混み合う時間帯でも止まりにくくなります。FLUX.1 [dev] のように **利用規約への同意が必要なモデル（ゲート付き）** を自分で試す場合は、トークンが必須です。1. huggingface.co で無料のアカウントを作る 2. 右上のアイコン →[Settings]→[Access Tokens]→[Create new token]。種類は **Read** を選び、作成された `hf_...` の文字列をコピー 3. 設定する - **Colab**：左の鍵のアイコン（シークレット）→[新しいシークレットを追加]→ 名前 `HF_TOKEN`、値に `hf_...` を貼り付け →「ノートブックからのアクセス」をオン。実行中にアクセスの許可を求められたら[アクセスを許可] - **ローカル**：`.venv` を有効にした PowerShell で `hf auth login` を実行し、`hf_...` を貼り付け（1 回だけ。以後は自動で使われる） 4. ゲート付きモデルは、さらにモデルのページで[Agree and access repository]を押して利用規約に同意する

注意：トークンはパスワードと同じです。ノートブックのセルに直接書かないでください。ノートブックを共有したり GitHub に上げたりするときに含めないこと。漏れたら Settings → Access Tokens で無効にできます。

B-3. モデルのダウンロード先を変える（C ドライブの空きが足りない場合だけ）

Hugging Face のモデルは既定で `C:\Users\<ユーザー>\.cache\huggingface` に保存されます（数十GBになります）。

```
setx HF_HOME D:\hf_cache          # 次回起動時から有効
```

B-4. オフライン（閉域網）で使う

1. インターネットに接続できるPCで一度ノートブックを実行し、モデルをキャッシュさせる
2. `HF_HOME` フォルダ（と `weights` フォルダ）を閉域網のPCへコピー
3. 閉域網側で次を設定してから実行：

```
setx HF_HUB_OFFLINE 1
setx TRANSFORMERS_OFFLINE 1
```

B-5. 最小のコード例（超解像、Real-ESRGAN）

```
import torch, spandrel
from PIL import Image
import numpy as np

model = spandrel.ModelLoader().load_from_file("RealESRGAN_x4plus.pth").cuda().eval()
img = np.asarray(Image.open("input.jpg").convert("RGB"), dtype=np.float32) / 255
x = torch.from_numpy(img).permute(2, 0, 1)[None].cuda()
with torch.no_grad():
    y = model(x)[0].clamp(0, 1).permute(1, 2, 0).cpu().numpy()
Image.fromarray((y * 255).round().astype(np.uint8)).save("output_x4.png")
```

B-6. ComfyUI をPythonから操作する（バッチ処理）

ComfyUI の画面で **Workflow → Export (API)** で保存した JSON を、HTTP で投げるとバッチ処理できます（ノートブック 15 の `run_workflow()` 関数を参照）。

C. 講師の PC の動作環境（全ノートブック・ワークフローの動作確認に使った構成）

同じ構成にそろえれば、配布したノートブックの結果（`notebooks_executed/`）とほぼ同じ結果が再現できます。

C-1. ハードウェア・OS

項目	内容
PC	HP OMEN MAX Gaming Laptop 16（ノート PC）
OS	Windows 11 Home（バージョン 10.0.26200）
CPU	Intel Core Ultra 9 275HX（24 コア）
メモリ（RAM）	64 GB
GPU	NVIDIA GeForce RTX 5090 Laptop GPU（VRAM 24 GB）
NVIDIA ドライバ	591.44（CUDA 13.1 まで対応）
ストレージ（使用量）	SSD。B. の Python 環境は約 45 GB（Hugging Face のモデル 38 GB、仮想環境 5 GB を含む）。A. の ComfyUI は、講座以外のモデルも含めて約 190 GB

C-2. Python 環境（B. のノートブック用）

項目	版
Python	3.12.11（uv 0.11.7 で作成した仮想環境）
PyTorch / torchvision	2.11.0 / 0.26.0（CUDA 12.8 版、cuDNN 9.19）
生成 AI	diffusers 0.42.1、transformers 5.21.0、accelerate 1.15.0、peft 0.21.2、safetensors 0.8.0、huggingface-hub 2.3.0
復元モデルの読み込み	spandrel 0.4.2
画質評価	lpips 0.1.4、torchmetrics 1.9.0、piq 0.8.0
画像処理・数値計算	opencv-python-headless 5.0.0.93、scikit-image 0.26.0、scipy 1.18.1、numpy 2.2.6、Pillow 13.0.0、matplotlib 3.11.2
軽量化（ノートブック 09）	onnx 1.23.2、onnxruntime 1.31.1、onnxscript 0.7.2
その他	psutil 7.2.2、jupyterlab 4.6.4

同じ環境の作り方（B-1 と同じ手順を、版を指定して実行するもの）：

```
# 教材を取得して、そのフォルダに入る（日本語・空白を含まない場所）
git clone https://github.com/takeshi-nagata-mgu/genai-image-restoration-seminar C:\genai_seminar
cd C:\genai_seminar

uv venv --python 3.12
.venv\Scripts\activate
uv pip install torch==2.11.0 torchvision==0.26.0 --index-url https://download.pytorch.org/whl/cu12
8
uv pip install -r requirements.txt

# 確認：2.11.0+cu128 True NVIDIA GeForce RTX 5090 Laptop GPU と出れば OK
python -c "import torch; print(torch.__version__, torch.cuda.is_available(), torch.cuda.get_device
_name(0))"

# ノートブックを開く（学習済みの重みは C:\genai_seminar\notebooks\weights に保存される）
jupyter lab
```

- ノートブック 14 は transformers 4.57 以上が必要です（Qwen3-VL のため）。上の版なら満たしています。
- ノートブック 15（SUPIR）は、ノートブックの中で ComfyUI と ComfyUI-SUPIR を作業フォルダにダウンロードし、それぞれの `requirements.txt` を入れます。

C-3. ComfyUI 環境（A. のワークフロー用）

項目	版
ComfyUI	0.33.1（Windows ポータブル版）
同梱の Python / PyTorch	3.12.10 / 2.13.0（CUDA 13.0 版）
使ったカスタムノード	ComfyUI-Manager、ComfyUI-SUPIR（kijai 版）、本講座の seminar_nodes（DifFace）

- ポータブル版の Python と PyTorch は、上の B. の仮想環境とは別物です（A-1 を参照）。両方を同じ PC に入れても干渉しません。

よくあるトラブル

症状	対処
`torch.cuda.is_available()` が False	CPU 版 PyTorch が入っている → CUDA 版を入れ直す。ドライバ更新
`CUDA out of memory`	画像サイズを小さく、タイル処理、FP16、`pipe.enable_model_cpu_offload()`
RTX 50 系で `no kernel image is available`	PyTorch を cu128 以降のビルドに更新
日本語パスでエラー	作業フォルダ・モデルフォルダを英数字のパスにする
Hugging Face で 401/403	モデルページで利用規約に同意 → `hf auth login`（ゲート付きモデルのみ。B-2b を参照）
`NameError: name '...' is not defined`	前のセルを実行していない／カーネルが再起動した → 最初のセルから順に実行し直す
ダウンロードが `429 Too Many Requests` で止まる	Hugging Face の回数制限 → トークンを設定して（B-2b）、しばらく待ってから再実行

ライセンスの確認（商用利用の前に必ず）

モデル	ライセンスの概要（必ず原文を確認）
Real-ESRGAN, SwinIR, SCUNet, DRUNet, LaMa, NAFNet, GFPGAN, DiffBIR	BSD / Apache / MIT 系（商用可が多い）
Stable Diffusion 1.5 / 2.x / SDXL	CreativeML OpenRAIL-M / ++（利用制限条項あり、商用可）
Stable Diffusion 3 / 3.5	Stability AI Community License（年商100万ドル未満は商用可。超えると有償契約）
SUPIR	**非商用**（研究目的。商用は別途許諾が必要）
CodeFormer, StableSR, ResShift, DifFace	**非商用**（S-Lab License）
RePaint, SinSR	**非商用**（CC BY-NC-SA）
FLUX.1-dev / Kontext-dev / FLUX.2-dev	**非商用ライセンス**（出力の商用利用条件に注意）
FLUX.1-schnell, FLUX.2 klein 4B, Qwen-Image 系	Apache 2.0
Qwen2.5-VL-3B（ノートブック 11）	**Qwen Research License**（研究用。7B 版は Apache 2.0）
pyiqa（画質評価ライブラリ）	**非商用**（PolyForm Noncommercial）
Qwen3-VL-4B（ノートブック 14）, Depth Anything V2 Small（ノートブック 13）	Apache 2.0
MVTec AD データセット	CC BY-NC-SA（非商用）
VisA データセット（ノートブック 14）	CC BY 4.0（出典表示で商用可）

（2026年10月に各配布ページで確認。採用前に必ず最新の原文を確認してください）